

A **CSS reset** is a set of CSS rules used to remove or reduce the browser's default styling.

Browsers automatically apply styles to HTML elements. For example:

- `<h1>` has a large font and top/bottom margins.
- `<p>` has default margins.
- `<ul>` has indentation and bullets.
- `<body>` usually has an 8px margin.

Because browsers may apply slightly different default styles, a CSS reset gives you a more consistent starting point.

Simple example

Without a reset:

```
<h1>Hello</h1>
<p>This is a paragraph.</p>
```

The browser automatically adds spacing around the heading and paragraph.

A basic reset might be:

```
* {
  margin: 0;
  padding: 0;
}
```

The `*` selector means:

Select every HTML element.

This removes default margins and padding from all elements.

Common basic reset

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

margin: 0

Removes outside spacing.

padding: 0

Removes inside spacing.

box-sizing: border-box

Makes width and height calculations easier.

For example:

```
div {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
}
```

With the default `content-box`, the actual width becomes:

$300 + 40 \text{ padding} + 10 \text{ border} = 350\text{px}$

With:

```
box-sizing: border-box;
```

the entire element remains 300px wide.

More complete reset

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

```
html,
body,
h1,
h2,
h3,
p,
ul,
ol {
  margin: 0;
  padding: 0;
}
```

```
ul,
ol {
  list-style: none;
}
```

```
img {
  max-width: 100%;
  display: block;
}
```

```
button,
input,
textarea,
select {
  font: inherit;
}
```

What these rules do

```
ul,
ol {
  list-style: none;
}
```

Removes bullets and numbering.

```
img {  
    max-width: 100%;  
    display: block;  
}
```

Prevents images from becoming wider than their container and removes the small inline-image gap.

```
button,  
input,  
textarea,  
select {  
    font: inherit;  
}
```

Makes form controls use the same font as the surrounding page.

Example in a full page

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <title>CSS Reset Example</title>  
  
    <style>  
        * {  
            margin: 0;  
            padding: 0;  
            box-sizing: border-box;  
        }  
  
        body {  
            font-family: Arial, sans-serif;  
            padding: 20px;  
        }  
  
        h1 {  
            margin-bottom: 10px;  
        }  
  
        p {  
            margin-bottom: 15px;  
        }  
    </style>  
</head>  
  
<body>  
    <h1>Hello World</h1>  
    <p>This page uses a basic CSS reset.</p>  
</body>  
</html>
```

Notice that after resetting the browser styles, you add back the spacing you actually want:

```
h1 {  
    margin-bottom: 10px;  
}
```

Reset versus Normalize

A **reset** removes most browser defaults:

```
* {
  margin: 0;
  padding: 0;
}
```

A **normalize stylesheet** keeps useful browser defaults but makes them more consistent across browsers.

So the difference is:

- **Reset:** removes default styles.
- **Normalize:** standardizes default styles.

Important caution

This reset:

```
* {
  margin: 0;
  padding: 0;
}
```

is simple, but it can remove useful formatting from lists, form controls, headings, and other elements. You must then define those styles yourself.

A practical beginner reset is:

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
}

img {
  max-width: 100%;
  display: block;
}
```

This removes the body margin, improves sizing behavior, and makes images responsive without removing every useful browser default.

`*::before, *::after`

explain ::

In CSS, `::` introduces a **pseudo-element**.

In this reset:

```
*  
*,  
*::before,  
*::after {  
    box-sizing: border-box;  
}
```

the selectors mean:

*

Select every actual HTML element.

*::before

Select the ::before pseudo-element of every element.

*::after

Select the ::after pseudo-element of every element.

What is a pseudo-element?

A pseudo-element is a CSS-generated part of an element. It is not written as a separate HTML tag.

For example:

```
<p class="note">Important message</p>  
.note::before {  
    content: "Note: ";  
    font-weight: bold;  
}
```

The page displays:

Note: Important message

But the word `Note:` is not inside the HTML. CSS creates it using `::before`.

::before

`::before` inserts generated content before the element's existing content.

```
p::before {  
    content: "→ ";  
}  
<p>Continue reading</p>
```

Displayed result:

→ Continue reading

::after

`::after` inserts generated content after the element's existing content.

```
p::after {
```

```
    content: " ✓";  
}
```

Displayed result:

[Continue reading ✓](#)

Why include them in a CSS reset?

Consider:

```
* {  
    box-sizing: border-box;  
}
```

This applies `border-box` to normal HTML elements, but generated `::before` and `::after` content may not receive it consistently through this rule alone.

Therefore, developers often write:

```
*,  
*::before,  
*::after {  
    box-sizing: border-box;  
}
```

This ensures that actual elements and their generated pseudo-elements all use the same box-sizing model.

Example with a pseudo-element box

```
.card::before {  
    content: "";  
    width: 50px;  
    height: 50px;  
    padding: 10px;  
    border: 5px solid black;  
}
```

Applying:

```
*,  
*::before,  
*::after {  
    box-sizing: border-box;  
}
```

makes the pseudo-element's total width remain 50px, including its padding and border.

Why two colons?

Modern CSS uses:

```
::before  
::after
```

for pseudo-elements.

A single colon is generally used for pseudo-classes:

```
:hover  
:focus  
:first-child
```

For example:

```
button:hover {  
    background-color: yellow;  
}
```

Here, `:hover` describes a state of the existing button.

By contrast:

```
button::before {  
    content: "★ ";  
}
```

creates a generated part of the button.

A useful distinction is:

```
:    pseudo-class – state or condition  
::   pseudo-element – generated part of an element
```